

The xmeaning package*

Valentin Dao†

Released 2025-12-18

Abstract

The *xmeaning* package’s main objective is to enhance the `\meaning` primitive by enlarging the amount of given information. It requires *expl3* as well as *pkginfograb*. Please note that this package is considered to be highly experimental for the moment.

Limitations

The `\xmeaning` macro will fail under certain precise circumstances:

1. If the parameter text *or* the replacement text of a macro contains “->”.
2. If you use `\xmeaning` for a constant defined using `\chardef` or `\mathchardef`.

Whether the compilation stops or succeeds, but with erroneous results, completely depends on what the argument is.

`\xmeaning` `\xmeaning* {⟨list of tokens⟩}`

On its simplest form, `\xmeaning` has a strictly similar usage to that of `\meaning`. The main argument can either correspond to a character or a control sequence. With *xmeaning*, it is possible to specify a list of tokens to display all their meanings, avoiding the need to repeat the macro. Unlike the T_EX primitive, which doesn’t always provide useful information, `\xmeaning` is capable of recognising 5 *token classes* among:

Characters A unique token.

Character definitions A control sequence declared with `\chardef` or `\mathchardef`.

T_EX registers Control sequences representing boxes, counts...

expl3 data types All the variable types implemented by L^AT_EX3.

Macros A macro, or any control sequence that doesn’t fall into the previous classes.

By *class*, we mean that the macro will produce an output adapted to each token type. Its starred variant will display the meaning of all auxiliary macros as described later.

*This file describes vv0.2.0, published 2026-02-10.

†E-mail: vdao.texdev@gmail.com

1 Characters

Unlike the `\meaning` primitive, which gives us only little information regarding characters, the package lists all there is to know about them.

- Character code, with its ASCII, Unicode, octal and hexadecimal representations.
- Category codes.
- Upper or lower case code (depending on the case of the queried character)[†].
- Space factor code[†].
- Delimiter code.
- Math code[†].

[†] These codes will be displayed only if they are not null.

For math code, the package goes beyond simply displaying it, as it analyses it according to its functioning as described in the `TEXbook`. The same applies to the delimiter code if it is not 0. Furthermore, it is not necessary to escape special characters with a backslash. The macro may produce incorrect results if you pass it an argument that is not recognised by the encoding. If you are using an engine other than `XTLATEX` or `LuaATEX`, keep this in mind.

`LATEX` Code:

```
\xmeaning o
```

`LATEX` Result:

Glyph	= the letter o
Dimensions	= width: 5.25pt
	height: 4.4pt
	depth: 0.06pt
Category code	= 11
Character code	= 111
Unicode	= U+006F
Octal	= 157
Hexadecimal	= 6f
Uppercase code	= 79 (0)
Space factor code	= 1000
Math code	= 1e0006f
	Class: Variable family
	Math family: 1
	Font position: 111

2 Character definitions

The package calls any character defined by `\chardef` and `\mathchardef` a *character definition*. The advantage over the primitive `\meaning` is that here the glyph is produced, allowing you to see the concrete result of the control sequence. It is precisely for this reason that constants defined with these two macros do not work (for example, certain plain`TEX` constants such as `\z@`, `\@M...`). They correspond to numbers and not to real

characters. Since the package cannot tell the difference, it will attempt to typeset a glyph with a character code that is certainly outside those defined by the font table. It also indicates whether the character should be used in text mode or math mode.

LaTeX Code:

```
\xmeaning{\%, \gamma}
```

LaTeX Result:

```
Character definition = \%
                  Glyph = %
                  Mode = text
```

```
Character definition = \gamma
                  Glyph =  $\gamma$ 
                  Mode = math
```

3 Registers

For registers, the package simply displays its type as well as its content.

LaTeX Code:

```
\newtoks\temptoks
\temptoks={Token register}
\xmeaning\temptoks
```

LaTeX Result:

```
Register name = \temptoks
          Type = token (no. 35)
          Content = Token register
```

4 Data types

LaTeX3 data types are much more complex than registers. That is why a few more entries compared to registers are present. `\xmeaning` will fail if the naming conventions for data types are not followed.

LaTeX Code:

```
\ExplSyntaxOn
\tl_new:N \l_my_token_tl
\tl_set:Nn \l_my_token_tl { Token~data~type }
\xmeaning\l_my_token_tl
\ExplSyntaxOff
```

LaTeX Result:

```
Variable = \l_my_token_tl
Data type = token
Content = Token data type
Scope = local
Use = public
```

5 Macros

The package produces a very complete output for macros, where the following information is typeset:

- The name of the macro.
- The type of macro.
- The prefix of the macro (`\long`, `\protected`, and `\outer`)[†].
- The parameter text[†].
- The replacement text[†].

[†]Here too, these elements are only displayed if they are present. For primitives, they will always be absent.

The macro *type* can be one of the following:

- `\TeX` primitive.
- `\TeX` macro.
- `expl3` primitive.
- `expl3` macro.
- argument specifier `expl3`, with distinction of the *unbraced* variants.
- `expl3` identity macro.
- `xparse` macro.

LaTeX Code:

```
\xmeaning{\kern, \newcommand}
```

LaTeX Result:

```
Control sequence = \kern
                  Type = TeX primitive

Control sequence = \newcommand
                  Type = TeX macro

Replacement text = \@star@or@long \new@command
```

6 Exhaustive meaning

With the starred variant of `\xmeaning`, it is possible to ask the package to display the meaning of all the auxiliary macros that make up the main argument. There is, however, no magical formula for knowing in advance what these auxiliary macros will be. The package therefore assumes that standard programming conventions are followed and, using a brute-force method, recursively applies `\xmeaning` to all of the following macros:

- `\foo⟨⏟⟩`.

- `\foo<code>` (for macros defined with *xparse*).
- `\foo<A-Z>` (`\fooA`, `\fooB...`).
- `<A-Z>\foo` (`\Afoo`, `\Bfoo...`).
- Nesting of `@` (`\@foo`, `\foo@...`).
- Replacement of *o* and *a* by `@` (`\f@@`, `\b@r`).
- `\foo<@Roman numeral>` (`\foo@ii`).
- `<_>\foo` (for internal macros of *expl3*).

This method does not guarantee that if several macros are found, they are related to each other. Nor is it certain that auxiliary macros will be found even after applying all these rules, which are not absolute (after all, everyone is free to name their macros as they see fit).

LaTeX Code:

```
\xmeaning*{\newcommand, \integral}
```

LaTeX Result:

```
Control sequence = \newcommand
      Type = TeX macro
Replacement text = \@star@or@long \new@command

Control sequence = \@newcommand
      Type = TeX macro
      Parameters = #1[#2]
Replacement text = \kernel@ifnextchar [{\@xargdef #1[#2]}\@argdef #1[#2]}

Control sequence = \new@command
      Type = TeX macro
      Parameters = #1
Replacement text = \@testopt {\@newcommand #1}0

Control sequence = \integral
      Type = TeX macro
      Prefix = \protected
Replacement text = \__cmd_start:nNNnnn {0{m}}\integral \integral code {\__cmd_grab_D:
      []\__cmd_grab_m_1:w }{\prg_do_nothing: }\c_novalue_tl }{}

Control sequence = \integral_code
      Type = xparse macro
      Prefix = \protected\long
      Parameters = #1#2
Replacement text = \group_begin: \tl_if_empty:nF
      {#1}{\__intexgral_pkg_extract_first_key_name:n
      {#1}\__intexgral_pkg_parse_macro_keys:n {#1}}\tl_set:Nn
      \l__intexgral_pkg_integrand_tl {#2}\__intexgral_pkg_print_integral:
      \group_end:
```

7 Implementation

```

1 <*package>
2 <@@=xmeaning_pkg>
3
4 \NeedsTeXFormat{LaTeX2e}
5 \RequirePackage{expl3}[2025-05-14]
6 \RequirePackage{pkginfograb}
7 \pkginfograbProvidesExplPackage{xmeaning}
8 {
9     name           = {xmeaning} ,
10    author          = {Valentin Dao},
11    date             = {2026-02-10},
12    creation         = {2025-12-18},
13    version          = {v0.2.0} ,
14    description      = {Enhanced implementation of the \string\meaning\ primitive.}
15 }
16
17 \ifpackagelater{expl3}{2025-05-14}{ }
18 {
19     \PackageError { xmeaning } { Support~package-expl3~too~old }
20     {
21         expl3~version~too~old~
22         'xmeaning'~requires~a~version~later~than:~2025-05-14.
23         \MessageBreak
24         Loading~xmeaning~will~abort.
25     }
26     \tex_endinput:D
27 }
28
29 \cs_new:Npn \__xmeaning_pkg_char_value_delcode:n #1 {
30     \tex_number:D \tex_delcode:D #1 \scan_stop:
31 }
32
33 \cs_new_protected:Npn \__xmeaning_pkg_tex_halign:n #1 {
34     \tex_halign:D { #1 }
35 }
36
37 \cs_generate_variant:Nn \regex_if_match:nnTF { neTF }
38 \cs_generate_variant:Nn \regex_match_case:nn { ne }
39 \cs_generate_variant:Nn \regex_match_case:nnF { neF }
40 \cs_generate_variant:Nn \regex_extract_all:nnN { neN }
41 \cs_generate_variant:Nn \int_to_hex:n { e }
42 \cs_generate_variant:Nn \int_from_hex:n { e }
43 \cs_generate_variant:Nn \int_case:nn { Vn }
44 \cs_generate_variant:Nn \tl_range:nnn { nnV }
45 \cs_generate_variant:Nn \char_generate:nn { en }
46 \cs_generate_variant:Nn \token_if_cs:NTF { cTF }
47 \cs_generate_variant:Nn \clist_put_right:Nn { Nf }
48 \cs_generate_variant:Nn \int_compare_p:nNn { nNV }
49
50 \int_new:N \l__xmeaning_pkg_char_math_fam_int
51 \int_new:N \l__xmeaning_pkg_char_math_slot_int
52 \int_new:N \l__xmeaning_pkg_small_variant_family_int

```

```

53 \int_new:N \l__xmeaning_pkg_small_variant_position_int
54 \int_new:N \l__xmeaning_pkg_big_variant_family_int
55 \int_new:N \l__xmeaning_pkg_big_variant_position_int
56
57 \bool_new:N \l__xmeaning_pkg_undefined_cs_bool
58 \bool_set_false:N \l__xmeaning_pkg_undefined_cs_bool
59 \bool_new:N \l__xmeaning_pkg_exhaustive_meaning_bool
60 \bool_set_false:N \l__xmeaning_pkg_exhaustive_meaning_bool
61
62 \tl_new:N \l__xmeaning_pkg_register_type_tl
63
64 \skip_new:N \l__xmeaning_pkg_meaning_skip
65
66 \skip_set:Nn \l__xmeaning_pkg_meaning_skip { 6pt + 3pt - 2pt }
67
68 \cs_new_protected:Npn \__xmeaning_pkg_print_clist:c #1
69 {
70   \begin{enumerate}
71   \clist_map_inline:cn { #1 }
72   {
73     \item ##1
74   }
75   \end{enumerate}
76 }
77
78 \cs_new_protected:Npn \__xmeaning_pkg_print_seq:c #1
79 {
80   \begin{enumerate}
81   \seq_map_inline:cn { #1 }
82   {
83     \item ##1
84   }
85   \end{enumerate}
86 }
87
88 \cs_new_protected:Npn \__xmeaning_pkg_print_prop:c #1
89 {
90   \prop_map_inline:cn { #1 }
91   { ##1-->~##2\tex_par:D }
92 }
93
94 \cs_new_protected:Npn \__xmeaning_pkg_character_meaning:N #1 {
95   \int_set:Nn \l_tmpa_int { \int_eval:n { `#1 } }
96   \tl_set:Nn \l_tmpb_tl { \int_to_Hex:n { \l_tmpa_int } }
97   \tl_set:Nn \l__xmeaning_pkg_utf_eight_representation_tl
98   {
99     U+
100     \prg_replicate:nn { \int_eval:n { 4 - \tl_count:N \l_tmpb_tl } } { 0 }
101     \tl_use:N \l_tmpb_tl
102   }
103   \__xmeaning_pkg_tex_halign:n { \tex_hfil:D##&\c_space_tl=\c_space_tl\ vbox_top:n{\raggedright
104     Glyph & \token_to_meaning:N #1\tex_cr:D
105     Dimensions & width:~\tex_the:D\tex_fontcharwd:D\tex_font:D `#1\tex_par:D
106     height:~\tex_the:D\tex_fontcharht:D\tex_font:D `#1\tex_par:D

```

```

107 \dim_compare:nNnF { \tex_the:D\tex_fontchardp:D\tex_font:D `#1 } = { 0 pt }
108 { depth:~\tex_the:D\tex_fontchardp:D\tex_font:D `#1\tex_par:D }
109 \dim_compare:nNnF { \tex_the:D\tex_fontcharic:D\tex_font:D `#1 } = { 0 pt }
110 { depth:~\tex_the:D\tex_fontcharic:D\tex_font:D `#1\tex_par:D }
111 \tex_cr:D
112 Category~code & \token_to_catcode:N #1\tex_cr:D
113 Character~code & \int_use:N \l_tmpa_int\tex_cr:D
114 Unicode & \tl_use:N \l__xmeaning_pkg_utf_eight_representation_tl\tex_cr:D
115 Octal & \int_to_oct:n { \l_tmpa_int }\tex_cr:D
116 Hexadecimal & \int_to_hex:n { \l_tmpa_int }\tex_cr:D
117 \bool_if:nF
118 {
119   \int_compare_p:nNn { \char_value_uccode:n { `#1 } } = { 0 }
120   ||
121   \int_compare_p:nNV { \char_value_uccode:n { `#1 } } = \l_tmpa_int
122 }
123 {
124   Uppercase~code & \char_value_uccode:n { `#1 }~(\char_generate:nn { \char_value_uccode
125 }
126 \bool_if:nF
127 {
128   \int_compare_p:nNn { \char_value_lccode:n { `#1 } } = { 0 }
129   ||
130   \int_compare_p:nNV { \char_value_lccode:n { `#1 } } = \l_tmpa_int
131 }
132 {
133   Lowercase~code & \char_value_lccode:n { `#1 }~(\char_generate:nn { \char_value_lccode
134 }
135 Space~factor~code & \char_value_sfcode:n { `#1 } \tex_cr:D
136 Math~code & \int_to_hex:e { \char_value_mathcode:n { `#1 } }
137 \sys_if_engine luatex:TF
138 {
139   \int_set:Nn \l_tmpb_int { \Umathcharclass`#1 }
140   \int_set:Nn \l__xmeaning_pkg_char_math_fam_int { \Umathcharfam`#1 }
141   \int_set:Nn \l__xmeaning_pkg_char_math_slot_int { \Umathcharslot`#1 }
142 }
143 {
144   \exp_args:NNe \int_set:Nn \l_tmpb_int
145   { \tl_head:e { \int_to_hex:e { \char_value_mathcode:n { `#1 } } } }
146   \exp_args:NNe \int_set:Nn \l__xmeaning_pkg_char_math_fam_int
147   { \exp_args:Ne \tl_item:nn { \int_to_hex:e { \char_value_mathcode:n { `#1 } } } { 2
148   \exp_args:NNe \int_set:Nn \l__xmeaning_pkg_char_math_slot_int
149   {
150     \int_from_hex:e {
151     \exp_args:Ne
152     \tl_range:nnn
153     { \int_to_hex:e { \char_value_mathcode:n { `#1 } } }
154     { 3 }
155     { 4 }
156   }
157   }
158 } \tex_par:D
159 Class:~
160 \int_case:Vn \l_tmpb_int

```



```

161     {
162         { 0 } { Ordinary }
163         { 1 } { Large-operator }
164         { 2 } { Binary-operation }
165         { 3 } { Relation }
166         { 4 } { Opening }
167         { 5 } { Closing }
168         { 6 } { Punctuation }
169         { 7 } { Variable-family }
170     }\tex_par:D
171     Math-family:~\int_use:N \l__xmeaning_pkg_char_math_fam_int\tex_par:D
172     Font-position:~\int_use:N \l__xmeaning_pkg_char_math_slot_int\tex_cr:D
173     \int_compare:nNnF { \number\delcode`#1 } < { 0 }
174     {
175         Delimiter~code &
176         \int_set:Nn \l__xmeaning_pkg_small_variant_family_int
177         {
178             \tl_head:e {
179                 \__xmeaning_pkg_char_value_delcode:n { `#1 }
180             }
181         }
182         \exp_args:NNe
183         \int_set:Nn \l__xmeaning_pkg_small_variant_position_int
184         {
185             \exp_args:Ne
186             \tl_range:nnn
187             { \__xmeaning_pkg_char_value_delcode:n { `#1 } }
188             { 2 }
189             { 3 }
190         }
191         \exp_args:NNe
192         \int_set:Nn \l__xmeaning_pkg_big_variant_family_int
193         {
194             \exp_args:Ne
195             \tl_item:nn
196             { \__xmeaning_pkg_char_value_delcode:n { `#1 } }
197             { 4 }
198         }
199         \exp_args:NNe
200         \int_set:Nn \l__xmeaning_pkg_big_variant_position_int
201         {
202             \exp_args:Ne
203             \tl_range:nnn
204             { \__xmeaning_pkg_char_value_delcode:n { `#1 } }
205             { 5 }
206             { 6 }
207         }
208         \__xmeaning_pkg_char_value_delcode:n { `#1 } \tex_par:D
209         Small~variant~family:~\int_use:N \l__xmeaning_pkg_small_variant_family_int \tex_par:D
210         Small~variant~position:~\int_use:N \l__xmeaning_pkg_small_variant_position_int \tex_p
211         Big~variant~family:~\int_use:N \l__xmeaning_pkg_big_variant_family_int \tex_par:D
212         Big~variant~position:~\int_use:N \l__xmeaning_pkg_big_variant_position_int
213     }
214     \tex_crcr:D

```

```

215 }
216 }
217
218 \cs_new_protected:Npn \__xmeaning_pkg_register_meaning:n #1 {
219   \regex_extract_all:neN { \([a-z]+\)(\d+) } { \cs_meaning:c { #1 } } \l_tmpa_seq
220   \str_case_e:nnF { \seq_item:Nn \l_tmpa_seq { 2 } }
221   {
222     { toks } { \tl_set:Nn \l__xmeaning_pkg_register_type_tl { token } }
223     { dimen } { \tl_set:Nn \l__xmeaning_pkg_register_type_tl { dimension } }
224   }
225   { \tl_set:Ne \l__xmeaning_pkg_register_type_tl { \seq_item:Nn \l_tmpa_seq { 2 } } }
226   \__xmeaning_pkg_tex_halign:n { \tex_hfil:D##&\c_space_tl=\c_space_tl\ vbox_top:n{\raggedright}
227   Register~name & \c_backslash_str #1 \tex_cr:D
228   Type & \tl_use:N \l__xmeaning_pkg_register_type_tl (no.~\seq_item:Nn \l_tmpa_seq { 3 })\
229   Content & \exp_last_unbraced:Ne \tex_the:D { \use:c { #1 } }
230   \tex_crcr:D
231 }
232 }
233
234 \cs_new_protected:Npn \__xmeaning_pkg_data_type_meaning:n #1 {
235   \__xmeaning_pkg_tex_halign:n { \tex_hfil:D##&\c_space_tl=\c_space_tl\ vbox_top:n{\raggedright}
236   Variable & \slshape \str_use:N \c_backslash_str \tl_to_str:n { #1 } \tex_cr:D
237   Data~type & \str_case_e:nn { \seq_item:Nn \l_tmpb_seq { 4 } }
238   {
239     { tl } { token }
240     { str } { string }
241     { seq } { sequence }
242     { int } { integer }
243     { fp } { floating~point }
244     { flag } { flag }
245     { clist } { comma~list }
246     { prop } { property~list }
247     { dim } { dimension }
248     { intarray } { integer~array }
249     { fpararray } { floating~point~array }
250     { bitset } { bitset }
251     { cctab } { category~code~table }
252     { bool } { boolean }
253     { regex } { regular~expression }
254     { ior } { input/output~read~stream }
255     { iow } { input/output~write~stream }
256     { box } { box }
257     { coffin } { coffin }
258   } \tex_cr:D
259   Content &
260   \exp_last_unbraced:Ne
261   \scan_stop:
262   {
263     \str_case_e:nn { \seq_item:Nn \l_tmpb_seq { 4 } }
264     {
265       { tl } { \exp_not:N \tl_use:c }
266       { str } { \exp_not:N \str_use:c }
267       { seq } { \exp_not:N \__xmeaning_pkg_print_seq:c }
268       { int } { \exp_not:N \int_use:c }

```

```

269         { fp      } { \exp_not:N \fp_use:c }
270         { flag    } { \exp_not:N \flag_height:c }
271         { clist   } { \exp_not:N \__xmeaning_pkg_print_clist:c }
272         { prop    } { \exp_not:N \__xmeaning_pkg_print_prop:c }
273         { dim     } { \exp_not:N \dim_use:c }
274         { bitset  } { \exp_not:N \bitset_use:c }
275         { bool    } { \exp_not:N \bool_to_str:c }
276         { box     } { \exp_not:N \box_use:c }
277     }
278 }
279 { #1 }\tex_cr:D
280 Scope & \str_case_e:nn { \seq_item:Nn \l_tmpb_seq { 2 } }
281 {
282     { g } { global }
283     { l } { local }
284     { c } { constant }
285 } \tex_cr:D
286 Use & \int_case:nn { \int_eval:n { \tl_count:e { \seq_item:Nn \l_tmpb_seq { 3 } } } }
287 {
288     { 1 } { public }
289     { 2 } { private }
290 }
291 \tex_crcr:D
292 }
293 }
294
295 \cs_new_protected:Npn \__xmeaning_pkg_chardef_meaning:n #1 {
296     \regex_extract_all:neN { \(\math)?char"([0-9A-Z]+) } { \cs_meaning:c { #1 } } \l_tmpa_seq
297     \__xmeaning_pkg_tex_halign:n { \tex_hfil:D##&\c_space_tl=\c_space_tl\ vbox_top:n{\raggedright}
298     Character~definition & \c_backslash_str #1 \tex_cr:D
299     Glyph &
300     \tl_if_empty:eTF { \seq_item:Nn \l_tmpa_seq { 2 } }
301     { \char\exp_last_unbraced:Ne"{ \seq_item:Nn \l_tmpa_seq { 3 } } }
302     { $ \mathchar\exp_last_unbraced:Ne"{ \seq_item:Nn \l_tmpa_seq { 3 } } $ }
303     \tex_cr:D
304     Mode & \tl_if_empty:eTF { \seq_item:Nn \l_tmpa_seq { 2 } } { text } { math }
305     \tex_crcr:D
306 }
307 }
308
309 \cs_new_protected:Npn \__xmeaning_pkg_macro_meaning:n #1 {
310     \__xmeaning_pkg_tex_halign:n { \tex_hfil:D##&\c_space_tl=\c_space_tl\ vbox_top:n{\raggedright}
311     Control~sequence &
312     \tl_set:Nn \l_tmpa_tl { #1 }
313     \tl_replace_all:Nnn \l_tmpa_tl { ~ } { \textvisiblespace }
314     \slshape \str_use:N \c_backslash_str \tl_use:N \l_tmpa_tl \tex_cr:D
315     Type &
316     \exp_args:Nc \token_if_macro:NT { #1 }
317     {
318         \regex_match_case:nnF
319         {
320             { \w+: [NnVvcofexTFwD]* } { expl3~macro }
321             { :{2}[NnVvcofexTFwD]_unbraced } { expl3~argument~specifier~(unbraced~type) }
322             { :{2}[NnVvcofexTFwD]\Z } { expl3~argument~specifier }

```

```

323         { :{3} } { expl3-identity-macro }
324         { [a-zA-Z]\s code } { xparse-macro }
325     }
326     { #1 }
327     { \TeX\ macro }
328 }
329 \exp_args:Nc \token_if_primitive:NT { #1 }
330 {
331     \regex_if_match:nnTF { tex\_w+:D } { #1 }
332     { expl3-primitive }
333     { \TeX\ primitive }
334 }
335 \tex_cr:D
336 \exp_args:Nc \token_if_primitive:NF { #1 }
337 {
338     \tl_if_empty:eF { \exp_args:Nc \cs_prefix_spec:N { #1 } }
339     { Prefix & \exp_args:Nc \cs_prefix_spec:N { #1 } \tex_cr:D }
340 }
341 \exp_args:Nc \token_if_primitive:NF { #1 }
342 {
343     \tl_if_empty:eF { \exp_args:Nc \cs_parameter_spec:N { #1 } }
344     { Parameters & \exp_args:Nc \cs_parameter_spec:N { #1 } \tex_cr:D }
345 }
346 \exp_args:Nc \token_if_primitive:NF { #1 }
347 { Replacement-text & \cs_replacement_spec:c { #1 } }
348 \tex_crcr:D
349 }
350 }
351
352 \cs_new_protected:Npn \__xmeaning_pkg_control_sequence_meaning:n #1 {
353     \cs_if_exist:cTF { #1 }
354     {
355         \regex_extract_all:nnNTF { \A(g|l|c)_{1,2}\w*([a-z]*) } { #1 } \l_tmpb_seq
356         { \__xmeaning_pkg_data_type_meaning:n { #1 } }
357         {
358             \regex_match_case:neF
359             {
360                 { \\\(math)?char"([0-9A-Z]+) } { \__xmeaning_pkg_chardef_meaning:n { #1 } }
361                 { \\\[a-z]+\d+ } { \__xmeaning_pkg_register_meaning:n { #1 } }
362             }
363             { \cs_meaning:c { #1 } }
364             { \__xmeaning_pkg_macro_meaning:n { #1 } }
365         }
366     }
367     {
368         \bool_set_true:N \l__xmeaning_pkg_undefined_cs_bool
369         \bool_if:NF \l__xmeaning_pkg_exhaustive_meaning_bool
370         { Undefined }
371     }
372 }
373
374 \cs_new_protected:Npn \__xmeaning_pkg_xmeaning_ii:n #1 {
375     \group_begin:
376     \ttfamily

```

```

377 \frenchspacing
378 \clist_set:Nn \l_tmpa_clist { #1 }
379 \seq_set_from_clist:NN \l_tmpa_seq \l_tmpa_clist
380 \seq_map_inline:Nn \l_tmpa_seq
381 {
382   \token_if_cs:NTF ##1
383   { \exp_args:Ne \__xmeaning_pkg_control_sequence_meaning:n { \cs_to_str:N ##1 } }
384   { \__xmeaning_pkg_character_meaning:N ##1 }
385   \bool_if:NF \l__xmeaning_pkg_undefined_cs_bool
386   { \skip_vertical:N \l__xmeaning_pkg_meaning_skip }
387 }
388 \group_end:
389 }
390
391 \cs_new:Npn \__xmeaning_pkg_xmeaning_tl_map:nw #1#2#3 \q_stop
392 {
393   \str_if_eq:nnTF { #2 } { #1 }
394   { @ }
395   { #2 }
396   \tl_if_empty:nF { #3 }
397   { \__xmeaning_pkg_xmeaning_tl_map:nw { #1 } #3 \q_stop }
398 }
399
400 \cs_new:Npn \__xmeaning_pkg_xmeaning_replace_a_tokens:n #1
401 { \__xmeaning_tl_map:nw { a } #1 \q_stop }
402
403 \cs_new:Npn \__xmeaning_pkg_xmeaning_replace_o_tokens:n #1
404 { \__xmeaning_tl_map:nw { o } #1 \q_stop }
405
406 \cs_new_protected:Npn \__xmeaning_pkg_xmeaning_i:n #1 {
407   \group_begin:
408   \bool_set_true:N \l__xmeaning_pkg_exhaustive_meaning_bool
409   \ttfamily
410   \frenchspacing
411   \clist_set:Nn \l_tmpa_clist { #1 }
412   \clist_map_inline:Nn \l_tmpa_clist
413   {
414     \clist_clear:N \l_tmpb_clist
415     \tl_set:Ne \l_tmpa_tl
416     {
417       \token_if_cs:NTF ##1
418       { \cs_to_str:N ##1 }
419       { ##1 }
420     }
421     \clist_put_right:Nf \l_tmpb_clist { \l_tmpa_tl\c_space_tl }
422     \clist_put_right:Nf \l_tmpb_clist { \l_tmpa_tl\c_space_tl code}
423     \clist_put_right:Ne \l_tmpb_clist { end\l_tmpa_tl}
424     \exp_args:NNe \int_set:Nn \l_tmpa_int { \tl_count:V \l_tmpa_tl }
425     \int_step_inline:nn { 26 }
426     {
427       \clist_put_right:Ne \l_tmpb_clist { \int_to_Alph:n { ####1 } \l_tmpa_tl, }
428       \clist_put_right:Ne \l_tmpb_clist { \l_tmpa_tl \int_to_Alph:n { ####1 }, }
429     }
430     \int_step_inline:nn { 10 }

```

```

431     { \clist_put_right:Ne \l_tmpb_clist { \l_tmpa_tl @\int_to_roman:n { ####1 } } }
432 \int_step_inline:nnn { 0 } { \l_tmpa_int }
433 {
434     \clist_put_right:Ne \l_tmpb_clist
435     {
436         \exp_args:NV \tl_range:nnn { \l_tmpa_tl } { 1 } { ####1 }
437         @
438         \exp_args:NV \tl_range:nnV { \l_tmpa_tl } { \int_eval:n{ ####1+1 } } \l_tmpa_int
439     }
440 }
441 \tl_if_in:NnT \l_tmpa_tl { a }
442 {
443     \clist_put_right:Ne \l_tmpb_clist
444     { \exp_args:NV \__xmeaning_pkg_xmeaning_replace_a_tokens:n \l_tmpa_tl }
445 }
446 \tl_if_in:NnT \l_tmpa_tl { o }
447 {
448     \clist_put_right:Ne \l_tmpb_clist
449     { \exp_args:NV \__xmeaning_pkg_xmeaning_replace_o_tokens:n \l_tmpa_tl }
450 }
451 \clist_put_right:Ne \l_tmpb_clist
452 { __\l_tmpa_tl }
453 \__xmeaning_pkg_xmeaning_ii:n { ##1 }
454 \clist_map_inline:Nn \l_tmpb_clist
455 { \exp_args:Nc \__xmeaning_pkg_xmeaning_ii:n { ####1 } }
456 }
457 \group_end:
458 }
459
460 \NewDocumentCommand{\xmeaning}{ s m }{
461     \IfBooleanTF{#1}
462     { \__xmeaning_pkg_xmeaning_i:n { #2 } }
463     { \__xmeaning_pkg_xmeaning_ii:n { #2 } }
464 }
465
466 \ExplSyntaxOff

```